

Variables relevantes, filtrado y muestreo

Clase 06 — Laboratorio de Métodos Cuantitativos Aplicados a la Gestión

Manuel Fernández San Martín

Laboratorio de Métodos Cuantitativos Aplicados a la Gestión
Universidad de Buenos Aires

Primer cuatrimestre 2026

Obtener submuestras de una base con distintos métodos, y construir variables a partir de los datos.

¿Qué vamos a ver?

Tres partes, tres bases

- 1 **Muestreo básico** — simple, porcentual, por filtro, undersampling
- 2 **Muestreo dirigido** — estratificado, por montos, por conglomerados
- 3 **Variables y visualización** — crear columnas, resolver faltantes, graficar

La idea de fondo

Cada método responde a una pregunta distinta. Elegir el método **es** una decisión de análisis, no un detalle técnico.

Las bases que usamos

Base	Tamaño	Para qué
Default de tarjetas (UCI)	30.000 × 25	muestreo básico
Card transactions	fraude	muestreo dirigido
Pokémon	800 × 13	variables y gráficos

Herramientas

`pandas` `matplotlib` `seaborn`

¿Para qué tomar una submuestra?

Motivos prácticos

- Revisar casos **a mano**: auditoría, control de carga
- Probar una transformación pesada sin esperar la base entera
- Contactar clientes: cada llamada **cuesta**
- Entrenar un modelo con clases **equilibradas**

Lo que define el método

No es el tamaño: es **qué queremos que la muestra conserve**. La composición de un grupo, los casos raros, o las unidades completas.

Cada objetivo pide otro método

Si quiero...	Uso...
un retrato fiel de la base	aleatorio simple o porcentual
mirar un subconjunto clases equilibradas	muestreo por filtro undersampling
que un grupo esté sí o sí	estratificado
los casos extremos	muestreo por montos
unidades sin partir	conglomerados

Los seis aparecen en este notebook.

Parte 1 · La base: clientes de tarjeta de crédito

Qué trae

30.000 clientes y 25 columnas: límite de crédito (LIMIT_BAL), edad, historial de pagos, montos facturados y la variable objetivo `default payment next month`.

```
import pandas as pd

muestra1 = pd.read_excel(
    "default of credit card clients.xls", header=1)

print(muestra1.shape)    # (30000, 25)
```

header=1

El archivo trae una fila de título **antes** de los nombres de columna. Sin ese argumento, pandas toma el título como encabezado y todas las columnas quedan mal nombradas.

Dos verificaciones de rutina

`.shape` después de **cada** muestreo: es la forma más barata de confirmar que se tomó lo que se quería.
`.reset_index()` cuando la submuestra se va a exportar o compartir: sin eso quedan los índices saltados de la base original.

Sobre `reset_index()`

Los índices originales no son basura: dicen **de dónde** salió cada fila. Se resetea al entregar, no al trabajar.

Parte 1 · Muestreo aleatorio simple

Todas las filas con la misma chance

Se fija un tamaño n y pandas elige al azar.

```
# n = 15, sin reposición
submuestra1 = muestra1.sample(15)

# n = 25, con reposición
submuestra1_cr = muestra1.sample(25, replace=True)
```

Agregar `random_state`

Sin semilla, cada corrida da otra muestra. Si alguien tiene que **revisar** o **reproducir** el trabajo, hay que fijarla: `sample(15, random_state=42)`.

sin reposición



cada fila entra a lo sumo una vez

con reposición



una fila puede salir repetida

Cuál usar

Sin reposición para revisar casos: nadie audita dos veces el mismo cliente. **Con** reposición cuando la repetición es parte del método: remuestreo, simulación, bootstrap.

Parte 1 · Muestreo porcentual

Se fija una proporción, no un tamaño

frac toma un porcentaje de las filas. Sobre 30.000 registros, frac=0.2 devuelve 6.000.

```
# 20% de la base, sin reposición
submuestra2 = muestra1.sample(frac=0.2)

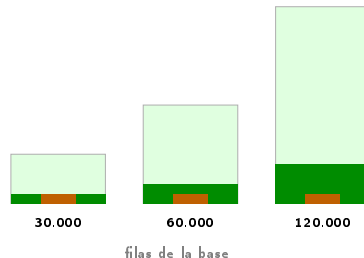
# 40% con reposición
submuestra2_cr = muestra1.sample(frac=0.4,
                                 replace=True)
```

n o frac

Con n el esfuerzo de revisión queda **fijo**. Con frac la muestra **crece** junto con la base.

verde: frac=0.2, crece con la base

naranja: n=6000, queda fijo



Regla práctica

n cuando el límite es la **capacidad de revisión**; frac cuando el objetivo es una **cobertura relativa**.

Parte 1 · Muestreo por filtro

Primero se restringe, después se muestrea

La condición define una **subpoblación**; el `sample` actúa solo adentro de ella.

```
# 10 clientes mayores de 50
submuestra3 = muestra1[
    muestra1['AGE'] > 50].sample(10)

# 25% de los que tienen limite entre 20k y 60k
submuestra3_2 = muestra1[
    (muestra1['LIMIT_BAL'] > 20000) &
    (muestra1['LIMIT_BAL'] < 60000)].sample(frac=0.25)
```

Cuidado con el paréntesis

Cada condición va entre paréntesis y se unen con `&` o `|`, no con `and` y `or`.



Lo que la muestra representa

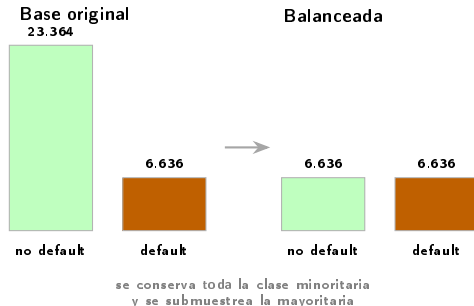
Ya **no representa a la base**: representa al subconjunto que definimos. Si el informe dice “los clientes”, cuando en realidad dice “los mayores de 50”, la conclusión queda mal atribuida.

Parte 1 · Undersampling: equilibrar las clases

El problema

La variable objetivo está desbalanceada: los clientes que entran en default son **minoría**. Una muestra aleatoria trae casi puros casos de la clase mayoritaria.

```
muestra1.groupby(  
    'default payment next month').count()['ID']  
  
m_true = muestra1[muestra1[  
    'default payment next month'] == 1]  
m_false = muestra1[muestra1[  
    'default payment next month'] == 0]  
  
balanceada = pd.concat(  
    [m_true, m_false.sample(m_true.shape[0])])
```



Lo que se pierde

La muestra balanceada **ya no refleja** la prevalencia real. Sirve para entrenar y comparar grupos, **no** para estimar cuántos clientes caen en default.

Parte 1 · Los cuatro métodos, comparados

Método	Cómo se pide	Cuándo	Qué hay que saber
Aleatorio simple	<code>sample(n)</code>	no hay estructura que preservar	puede perder grupos chicos
Porcentual	<code>sample(frac=)</code>	la muestra debe escalar con la base	la proporción es la decisión
Por filtro	<code>df[cond].sample()</code>	la pregunta es sobre un subconjunto	representa al subconjunto, no a la base
Undersampling	<code>concat + sample</code>	la clase objetivo está desbalanceada	rompe la prevalencia real a propósito

Lo que ningún muestreo arregla

La submuestra **hereda** los defectos de la base: duplicados, faltantes, errores de carga. Muestrear reduce volumen, no corrige calidad.

Lo que conviene dejar escrito

El método, el tamaño, la condición si hubo filtro y el `random_state`. Sin eso, la muestra no se puede reconstruir ni discutir.

Parte 2 · La base: transacciones con tarjeta

Qué trae

Transacciones con número de tarjeta (Cardnum), fecha, comercio (Merchnum), monto (Amount) y la marca Fraud.

```
data = pd.read_excel("../card_transactions.xlsx")

casos_ordenados = data.sort_values(
    by=['Cardnum', 'Date', 'Merchnum', 'Amount'],
    ascending=True).reset_index()

data['Fraud'].value_counts()
```

Por qué ordenar primero

Ordenar por tarjeta y fecha pone juntas las transacciones de un mismo plástico: así se ve la **secuencia** de consumo, que es donde aparece el patrón de fraude.

Por qué esta base pide otros métodos

El fraude es **raro** y no está repartido al azar: se concentra en **montos altos** y en **unas pocas tarjetas**. Un muestreo aleatorio simple casi no trae casos de fraude, y los que trae aparecen sueltos, sin el resto de la actividad de esa tarjeta.

Los tres métodos de esta parte

- **Estratificado:** garantizar ambas clases
- **Por montos:** ir a los casos extremos
- **Conglomerados:** traer la tarjeta completa

Parte 2 · Muestreo estratificado por fraude

Muestrear dentro de cada clase

Se toma un número fijo de cada categoría de Fraud, de modo que ambas queden representadas.

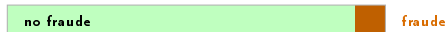
```
fraud = data[data['Fraud'] == 1].sample(
    n=min(1000, data['Fraud'].value_counts()[1]),
    random_state=42)

no_fraud = data[data['Fraud'] == 0].sample(
    n=min(1000, data['Fraud'].value_counts()[0]),
    random_state=42)
```

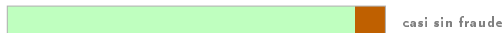
El min() no es decorativo

Si la clase minoritaria tiene menos de 1.000 casos, pedir 1.000 sin reposición **falla**. El min toma “1.000 o todos los que haya”.

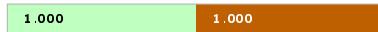
Población



Aleatorio simple



Estratificado



igual asignación: las dos clases pesan lo mismo a costa de la proporción real

Igual asignación, no proporcional

Con 1.000 de cada lado la muestra **no** conserva la prevalencia: sirve para **comparar** fraude contra no fraude, no para estimar cuánto fraude hay.

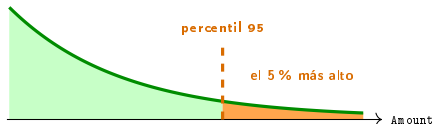
Parte 2 · Muestreo por montos: ir a los extremos

El umbral sale de los datos

No se elige un monto “alto” a ojo: se toma un **percentil** de la distribución observada.

```
umbral = submuestra_estratificada[
    'Amount'].quantile(0.95)

altos = submuestra_estratificada[
    submuestra_estratificada['Amount'] >= umbral]
```



El fraude suele concentrarse acá: la selección es deliberadamente no representativa.

Por qué un percentil y no un número fijo

Un umbral en pesos queda viejo con la inflación o cambia de sentido entre segmentos. El percentil se **recalcula** con los datos y siempre significa lo mismo: “el 5 % más alto”.

Buscar problemas no es retratar la población

Para auditoría está **bien** concentrarse en los casos extremos: sube la chance de encontrar algo. Lo que no se puede es presentar esa selección como si describiera al conjunto.

Parte 2 · Muestreo por conglomerados

La unidad es el grupo, no la fila

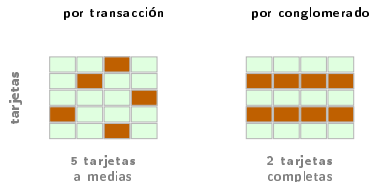
Se divide la población en **conglomerados** —acá, cada **Cardnum**— se eligen algunos y se toman **todas** sus transacciones.

```
card_numbers = altos['Cardnum'].unique()
conteo = altos['Cardnum'].value_counts()
sospechosas = conteo[conteo > 5].index.unique()

cluster = altos[
    altos['Cardnum'].isin(sospechosas)]
```

El criterio del ejemplo

No se eligen tarjetas al azar: se eligen las que tienen **más de 5** transacciones de alto valor. Es un conglomerado seleccionado **por sospecha**.



Por qué importa

El fraude se detecta mirando la **secuencia** de consumo de una tarjeta. Una transacción suelta, fuera de su contexto, no dice casi nada.

Parte 3 · Mirar la base antes de tocarla

Tres llamadas de reconocimiento

- `.info()` — filas, tipos, nulos y memoria.
- `.describe().transpose()` — media, desvío, percentiles, mínimo y máximo.
- `.isnull().sum()` — dónde faltan datos y cuántos.

```
archivo_datos = pd.read_csv(url)

archivo_datos.info()
archivo_datos.describe().transpose()
archivo_datos.isnull().sum()
```

Por qué `.transpose()`

Con muchas columnas, `describe()` se lee de costado. Transpuesto, cada variable es una fila y se compara de un vistazo.

La base Pokémon

800 filas y 13 columnas: nombre, dos tipos, y los atributos numéricos (HP, Attack, Defense, Speed, ...).

Lo que aparece al mirar

Una sola columna tiene faltantes: **Type 2**, con **386** nulos sobre 800. No es un error de carga: son los Pokémon que **tienen un solo tipo**. El faltante **significa algo**.

Parte 3 · Datos faltantes: qué significa el hueco

El procedimiento de tres pasos

1. Ver dónde faltan y cuántos son.
2. Resolver según lo que el hueco **significa**.
3. Volver a chequear.

```
archivo_datos.isnull().sum()      # Type 2: 386  
  
archivo_datos["Poder 2"] = (  
    archivo_datos["Poder 2"].fillna("Ninguno"))  
  
archivo_datos.isnull().sum()      # 0
```

“Ninguno” no es rellenar por rellenar

Acá el nulo no es información perdida: es **ausencia real** de segundo tipo. Por eso se reemplaza por una categoría con sentido, no por una media ni por un cero. Y el paso 3 confirma que efectivamente desaparecieron.

Tres huecos distintos

El hueco es	Significa	Qué hacer
no aplica	el atributo no existe	categoría propia
no se midió	falta el dato	imputar o dejar nulo
error de carga	el dato debería estar	corregir en origen

Parte 3 · Renombrar y seleccionar columnas

Renombrar con diccionario

`rename` recibe pares {viejo: nuevo}. Solo cambia las que se nombran; las demás quedan igual.

```
archivo_datos.rename(columns={
    "Name": "Nombre", "Speed": "Velocidad",
    "Type 1": "Poder 1", "HP": "Salud"},
    inplace=True)

archivo_datos.loc[:, ["Attack", "Defense",
                     "Sp. Atk", "Sp. Def"]]

archivo_datos["Poder 1"].unique()
```

`inplace=True`

Modifica el DataFrame **en el lugar** y no devuelve nada. Si se escribe `df = df.rename(..., inplace=True)`, la variable queda en `None` y se pierde la base.

`.unique()`: el chequeo previo

Antes de agrupar o filtrar por una columna de texto, `.unique()` muestra las categorías reales. Ahí aparecen las variantes de escritura, los espacios de más y las categorías que nadie esperaba.

En esta base, `Poder 1` tiene **18** categorías.

Parte 3 · De variable continua a categórica

.loc con condiciones explícitas

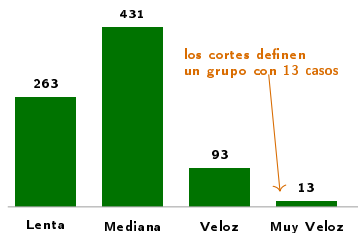
Cada condición asigna una etiqueta a las filas que la cumplen. Los cortes los elige **quien analiza**.

```
d.loc[d["Velocidad"] <= 50, 'Reaccion'] = "Lenta"
d.loc[(51 <= d["Velocidad"]) &
      (d["Velocidad"] <= 100), 'Reaccion'] = "Mediana"
d.loc[(101 <= d["Velocidad"]) &
      (d["Velocidad"] <= 130), 'Reaccion'] = "Veloz"
d.loc[d["Velocidad"] > 130, 'Reaccion'] = "Muy Veloz"
```

Los tramos tienen que cubrir todo

Si un rango queda afuera, esas filas quedan en NaN **sin aviso**. Conviene chequear con `.isnull().sum()` después de crear la columna.

Casos por tipo de reacción (n = 800)



El corte cambia la lectura

Con estos umbrales, “Muy Veloz” tiene 13 casos: no alcanza para sacar conclusiones sobre ese grupo. Si los tramos salieran de **cuantiles**, los cuatro grupos tendrían 200 casos cada uno — y otra interpretación.

Parte 3 · Operaciones entre columnas y funciones propias

Variables construidas

Una columna nueva puede combinar varias existentes. Acá, un índice de ataque como promedio de tres atributos.

```
d["Poder de Ataque"] = (  
    d["Salud"]*(1/3) + d["Attack"]*(1/3)  
    + d["Sp. Atk"]*(1/3))  
d["Poder de Ataque"] = round(d["Poder de Ataque"])  
  
def pokemon_poder(poder):  
    """Devuelve las filas de un poder dado"""  
    cond = d["Poder 1"].isin([poder])  
    return d.loc[cond, :]  
  
pokemon_poder("Grass")
```

La ponderación es una decisión

Promediar los tres atributos con peso 1/3 supone que **valen lo mismo**. Es un supuesto, no un cálculo neutral: con otros pesos cambia el ranking.

Envolver un filtro en una función

Cuando la misma consulta se repite, conviene una función: se escribe una vez, se documenta con docstring y se corrige en un solo lugar.

.isin() frente a ==

.isin([...]) acepta **varias** categorías a la vez y se lee mejor que encadenar comparaciones con |.

Parte 3 · Qué gráfico para qué pregunta

La pregunta manda

Si quiero ver...	Gráfico
cuántos hay de cada categoría	barras / countplot
cómo se reparte un total	torta
dos categorías cruzadas	barras apiladas
la forma de una variable	histograma
dispersión y valores extremos	boxplot
relación entre dos variables	dispersión
relaciones entre muchas	mapa de calor



barras



torta



histograma



boxplot



dispersión

En el notebook

```
plot(kind="bar", stacked=True, plot.pie(),  
boxplot(), hist(), plot(kind="scatter") y  
sb.heatmap(df.corr())).
```

El mapa de calor engancha con lo anterior

`corr()` muestra asociación, **no** causa. Sirve para decidir qué mirar después, no para concluir.

Ideas clave de la clase

Muestreo

- El método lo decide **qué tiene que conservar** la muestra, no el tamaño
- `n` fija el esfuerzo; `frac` fija la cobertura
- Filtrar antes de muestrear cambia **a quién** representa la muestra
- Undersampling y estratificado **rompen** la proporción real a propósito
- Conglomerados conserva la **unidad completa**

Variables y gráficos

- Antes de transformar: `.info()`, `.describe()`, `.isnull().sum()`
- Un faltante puede ser “no aplica”: ahí se le da **categoría propia**
- Los cortes de una categórica son una **decisión** y cambian la lectura
- El gráfico se elige por la **pregunta**

Lo que siempre se documenta

Método, tamaño, condición y `random_state`. Una muestra que no se puede reconstruir no se puede discutir.

Actividad práctica

Consigna

- 1 Cargar la base y reportar `shape`, tipos y faltantes.
- 2 Generar tres submuestras: simple con `n`, porcentual con `frac` y una por filtro. Comparar `shape` y justificar el filtro.
- 3 Verificar el balance de la variable objetivo y construir una muestra balanceada.
- 4 Crear una variable categórica a partir de una continua y reportar cuántos casos cae en cada tramo.
- 5 Elegir **dos** gráficos y explicar qué pregunta responde cada uno.

Requisitos de entrega

Todas las muestras con `random_state` fijo, y cada decisión (umbral, condición, tramos) escrita en una celda de texto.

Entregable

Notebook de Colab con el código comentado, los gráficos y una conclusión breve por punto.

Fuentes de datos

- UCI — *Default of credit card clients*
- *Card transactions* (fraude)
- *Pokemon.csv*

¡Gracias!

manuelfsm03@gmail.com

Laboratorio de Métodos Cuantitativos Aplicados a la Gestión — FCE UBA